

Design and Analysis of Algorithm

Lecture 17: Circular Linked List

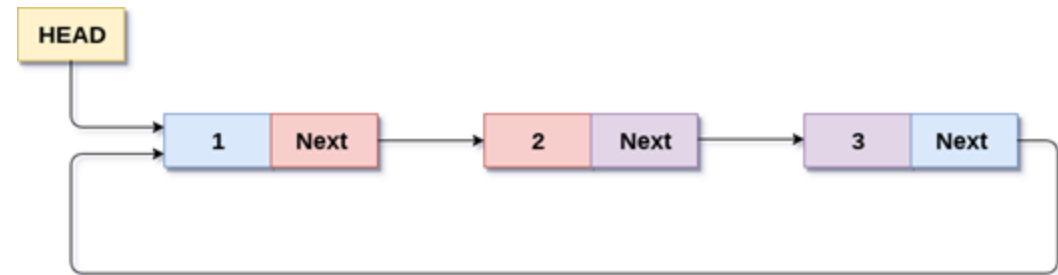
Content



- **Introduction**
- **Operations**

Circular Linked List

A circular linked list is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node.



This structure allows for continuous traversal without any interruptions.

Basic Operations

Following are the basic operations supported by a circular linked list.

Insertion

- Adds an element at the beginning of the list.

Deletion

- Deletes an element of the list.

Traversal

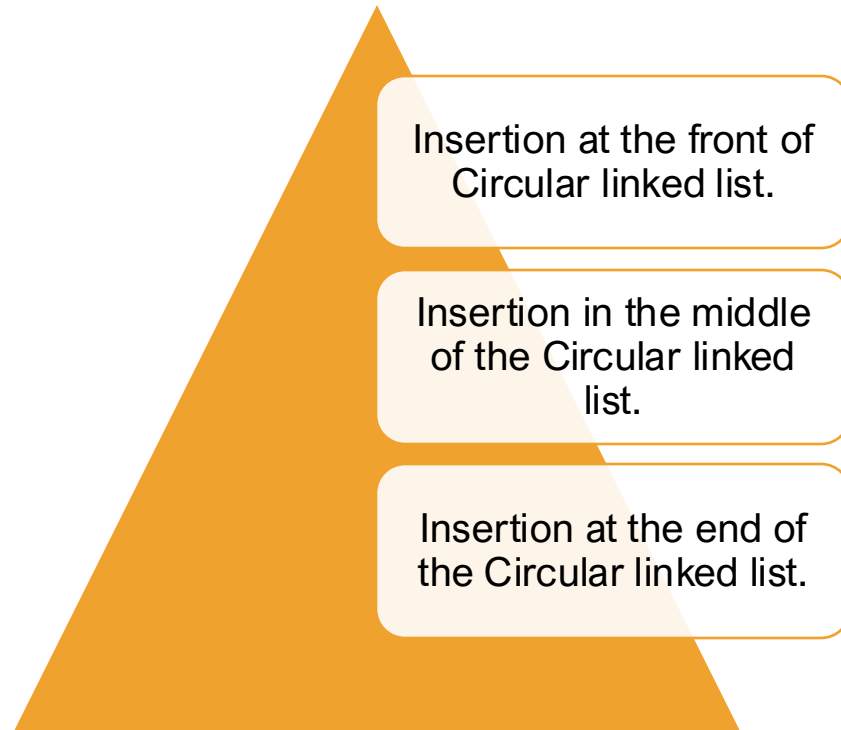
- Traverse the complete list.

Search

- Searches an element using the given key.

Insertion In Circular Linked List

There are three situation for inserting element in Circular linked list.



Insertion In Circular Linked List

Insertion at the front of Circular linked list.

Step1. Create the new node

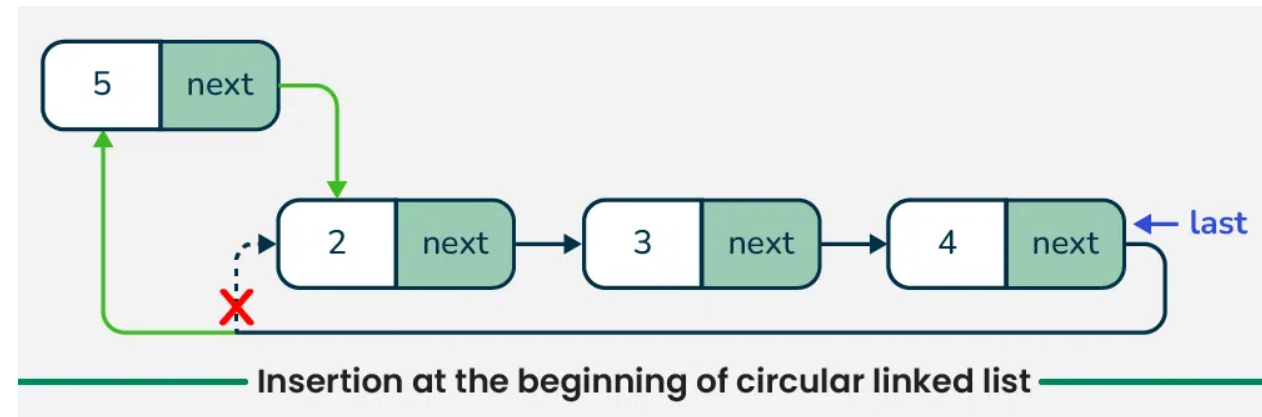
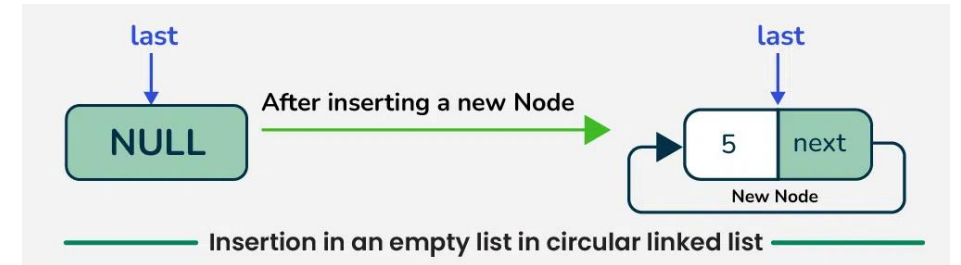
Step2. Set the new node's next to itself (circular!)

Step3. If the list is empty, return new node.

Step4. Set our new node's next to the front.

Step5. Set tail's next to our new node.

Step6. Return the end of the list.



Insertion In Circular Linked List

Pseudocode

```
INSFIRST(INFO, LINK, START, AVAIL, ITEM)
    If AVAIL = NULL, then:
        Write: "OVERFLOW" and Exit. // Overflow condition
    Set NEW := AVAIL
    Set AVAIL := LINK[AVAIL] // Remove first node from AVAIL list
    Set INFO[NEW] := ITEM // Store ITEM in the new node
    If START = NULL, then: // If list is empty, make the new node point to itself
        Set LINK[NEW] := NEW
        Set START := NEW
    Else:
        Set TEMP := START
        While LINK[TEMP] ≠ START, do:
            TEMP := LINK[TEMP] // Traverse to the last node
        Set LINK[NEW] := START // New node points to the first node
        Set LINK[TEMP] := NEW // Last node now points to the new node
        Set START := NEW // Update START to the new node
    Exit.
```

Insertion In Circular Linked List

Insertion at the end of Circular linked list

Step1. Create the new node

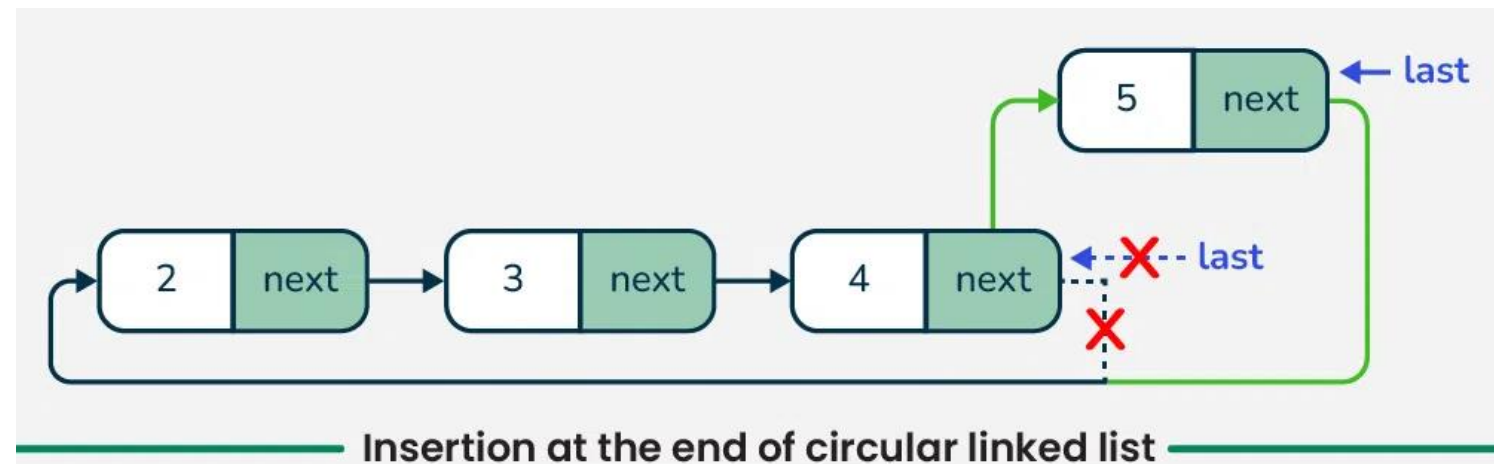
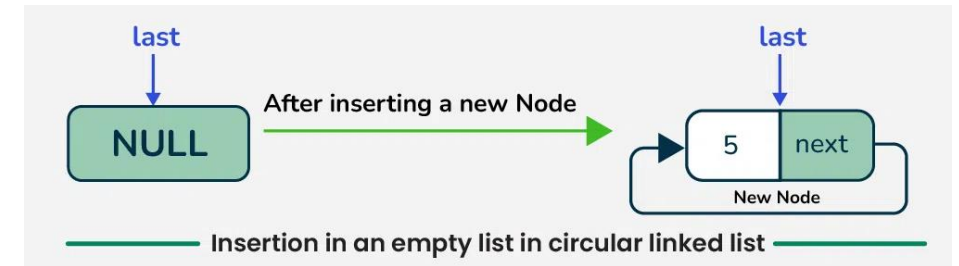
Step2. Set the new node's next to itself (circular!)

Step3. If the list is empty, return new node.

Step4. Set our new node's next to the front.

Step5. Set tail's next to our new node.

Step6. Return the end of the list.



Insertion In Circular Linked List

Algorithm

INSLAST(INFO, LINK, START, AVAIL, ITEM)

 If AVAIL = NULL, then:

 Write: "OVERFLOW" and Exit. // Overflow condition

 Set NEW := AVAIL

 Set AVAIL := LINK[AVAIL] // Remove first node from AVAIL list

 Set INFO[NEW] := ITEM // Store ITEM in the new node

 If START = NULL, then: // If list is empty, make the new node point to itself

 Set LINK[NEW] := NEW

 Set START := NEW

 Else:

 Set TEMP := START

 While LINK[TEMP] ≠ START, do:

 TEMP := LINK[TEMP] // Traverse to the last node

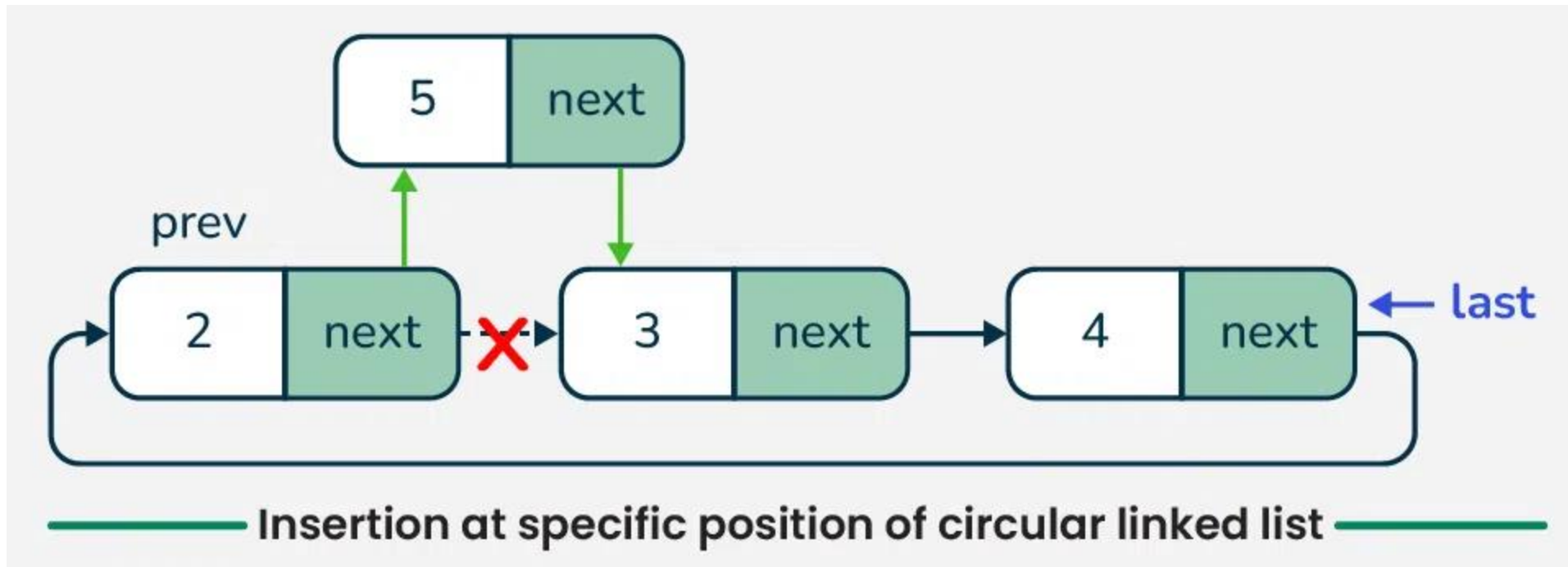
 Set LINK[TEMP] := NEW // Last node now points to the new node

 Set LINK[NEW] := START // New node points back to the first node

Exit.

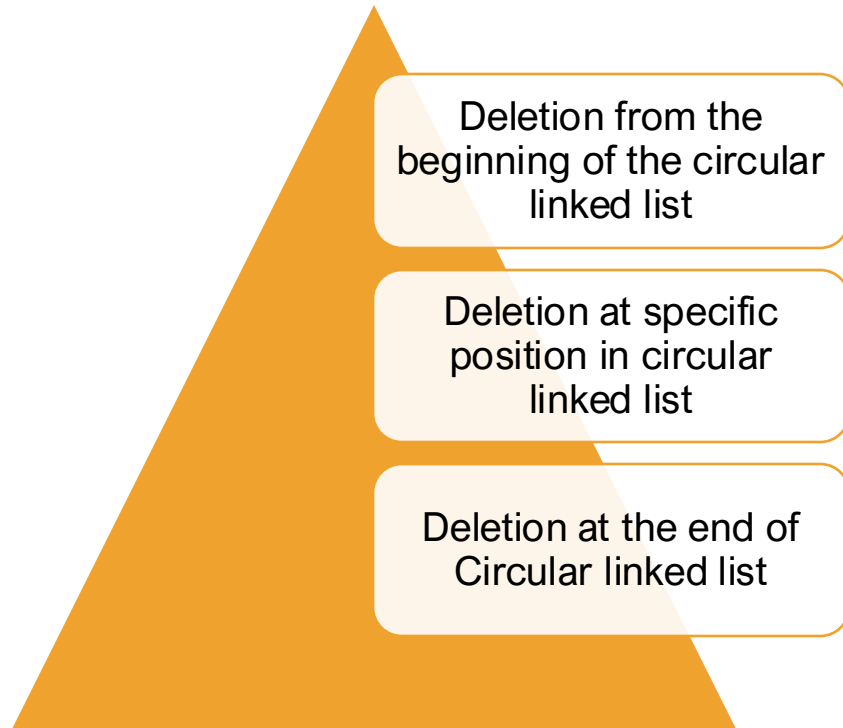
Insertion In Circular Linked List

Insertion at some other location in Circular linked list



Deletion In Circular Linked List

Deletion involves removing a node from the linked list. The main difference is that we need to ensure the list remains circular after the deletion.



Deletion In Circular Linked List

Deletion at the front of Circular linked list.

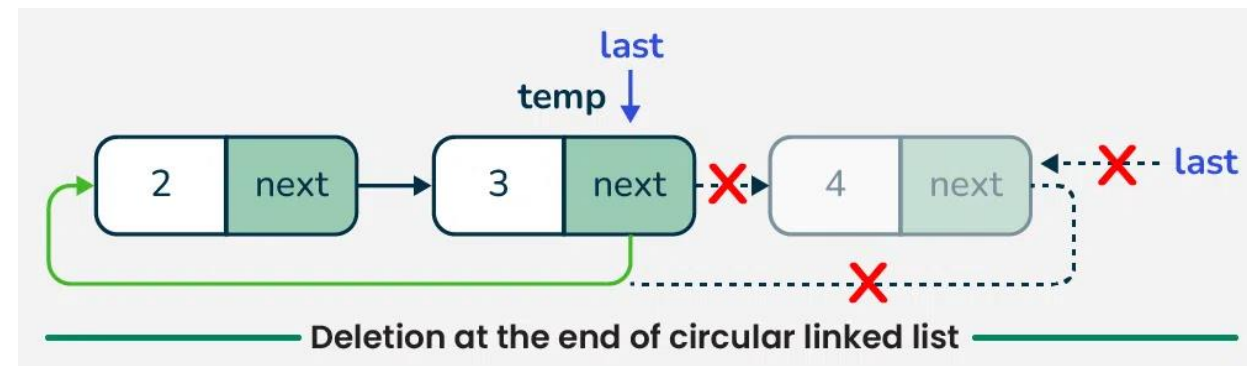
- Step 1. Check if the list is empty. If yes, return NULL (nothing to delete).
- Step 2. If the list contains only one node, delete it and return NULL.
- Step 3. Set the tail's next pointer to the second node (head's next).
- Step 4. Update the head pointer to point to the second node.
- Step 5. Free the memory of the old front node.
- Step 6. Return the updated head of the list.



Deletion In Circular Linked List

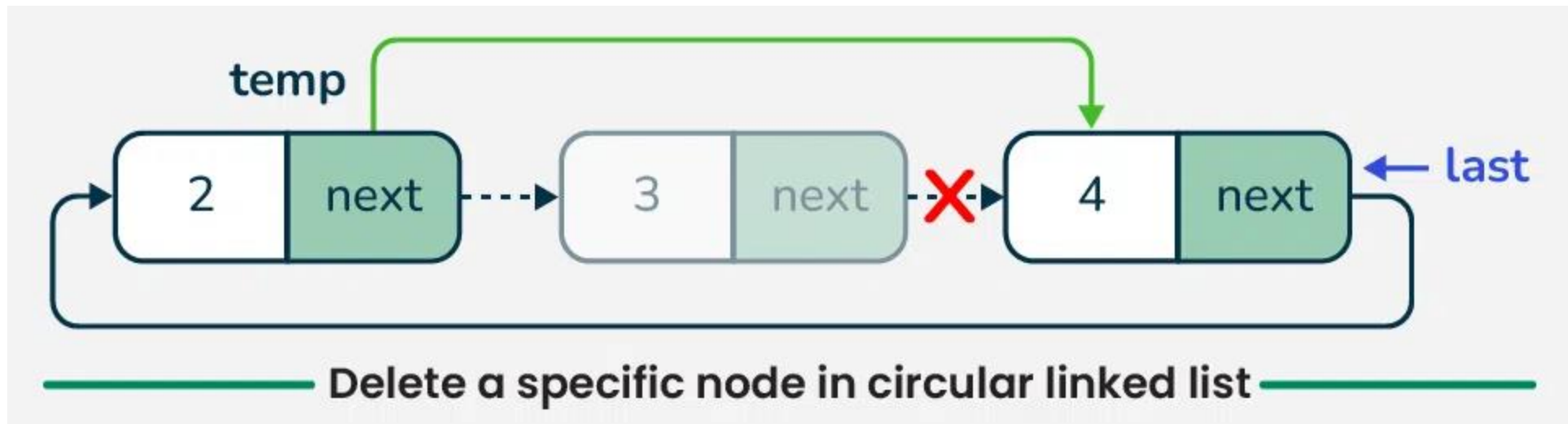
Deletion at the end of Circular linked list.

- Step 1. Check if the list is empty. If yes, return NULL (nothing to delete).
- Step 2. If the list contains only one node, delete it and return NULL.
- Step 3. Traverse the list to find the second-last node (the node before the tail).
- Step 4. Update the second-last node's next pointer to point to the head (maintaining circularity).
- Step 5. Update the tail pointer to the second-last node.
- Step 6. Free the memory of the old tail node.
- Step 7. Return the updated head of the list.



Deletion In Circular Linked List

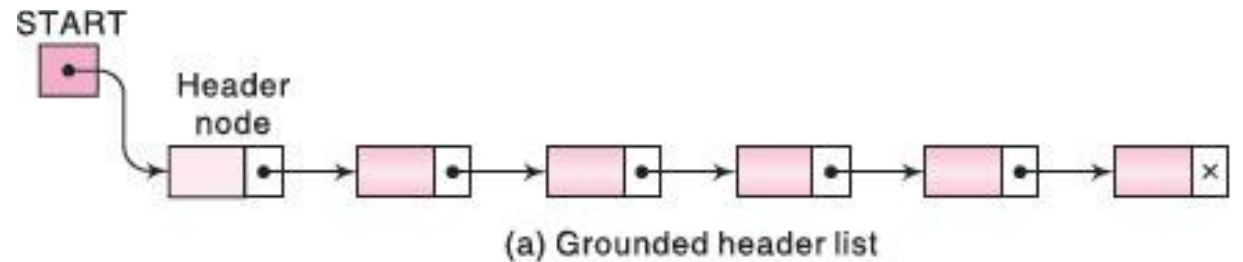
Deletion at specific position in circular linked list



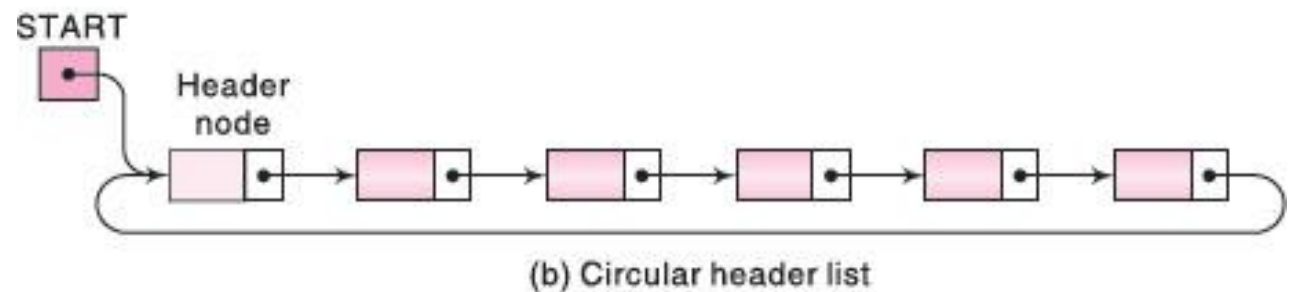
Header linked lists

A header linked list is a linked list which always contains a special node, called the header node, at the beginning of the list.

(1) A grounded header list is a header list where the last node contains the null pointer.



(2) A circular header list is a header list where the last node points back to the header node.

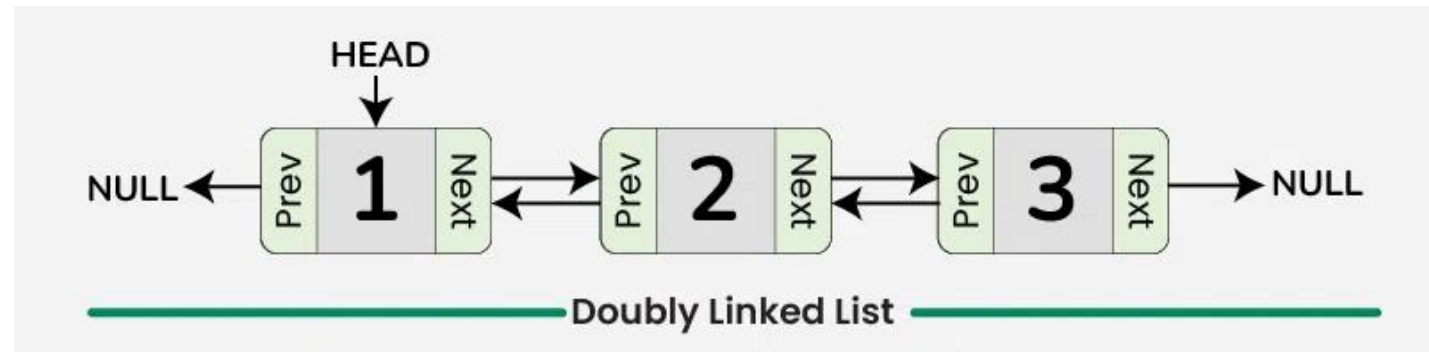


Doubly Linked List

A doubly linked list comprises a set of nodes, each node having a pointer to the next node in the list AND a pointer to the previous node.

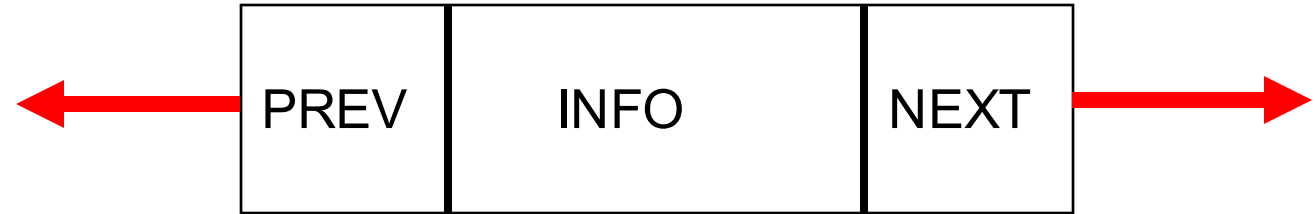
In a data structure, a doubly linked list is represented using nodes that have three fields:

- 1.Data
- 2.A pointer to the next node (**next**)
- 3.A pointer to the previous node (**prev**)



Node

```
typedef struct _node {  
    int contents ;  
    struct _node *next ;  
    struct _node *prev;  
} node ;
```



```
//Create the node  
struct node* n = malloc(sizeof(struct node));  
n->contents = 800;  
n->next = NULL; n->prev = NULL;
```

Doubly Linked List: Traversal

Traversal in a Doubly Linked List involves visiting each node, processing its data, and moving to the next or previous node using Traversal.

Forward Traversal

```
void forwardTraversal(struct Node* head) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        temp = temp->next;  
    }  
}
```

Backword Traversal

```
void backwordTraversal(struct Node* tail) {  
    struct Node* temp = tail;  
    while (temp != NULL) {  
        temp = temp->prev;  
    }  
}
```